T.A.R.K.A.S.

Áudio Espacial de cabo a rabo

Como o Dolby Atmos funciona, a ciência por trás dele e o projeto completo de um motor espacial próprio para o app Tarkas, com cálculo em tempo real por frequência

Documento técnico de pesquisa e arquitetura · versão 1.1 · 29 de outubro de 2026 (revisado após o protótipo da fase 0)

Sobre as fontes. Dolby Atmos é um sistema proprietário: o renderizador, os codecs e o algoritmo de agrupamento de objetos não são publicados. Este documento separa três tipos de informação: (1) o que está em normas públicas (ETSI, ITU-R, SMPTE, AES) e em documentação oficial de ferramentas; (2) ciência pública de psicoacústica e processamento de sinais, revisada por pares; (3) inferências e decisões de projeto nossas, sempre marcadas como tal. Nada aqui é código ou segredo da Dolby, e o motor proposto não usa o nome nem a tecnologia patenteada deles.

Sumário

1. Resumo executivo 4

2. O que é o Dolby Atmos: história e conceito 5

2.1 Linha do tempo . . . 5

2.2 Três formas de representar som 3D . . . 5

3. Arquitetura do Dolby Atmos 6

3.1 Beds, objetos e as 128 entradas . . . 6

3.2 Metadados de um objeto . . . 6

3.3 Do estúdio ao seu ouvido . . . 6

3.4 Os codecs . . . 7

4. Coordenadas e layouts de alto-falantes 8

4.1 Dois sistemas de coordenadas . . . 8

4.2 Layouts padronizados (ITU-R BS.2051) . . . 8

5. Psicoacústica: como o cérebro localiza sons 10

5.1 ITD – diferença de tempo entre os ouvidos . . . 10

5.2 ILD – diferença de nível entre os ouvidos . . . 10

5.3 Pistas espectrais da orelha (pinna) e elevação . . . 10

5.4 Distância, sala e precedência . . . 11

5.5 Resolução de frequência do ouvido . . . 11

6. A matemática do panning 12

6.1 Leis de pan entre dois alto-falantes . . . 12

6.2 VBAP – Vector Base Amplitude Panning (Pulkki, 1997) . . . 12

6.3 O panner do Atmos . . . 12

6.4 Gestão de graves . . . 12

7. Renderização binaural (fones) 13

7.1 HRTF e HRIR . . . 13

7.2 Bases de HRTF abertas (formato SOFA, AES69) . . . 13

7.3 Interpolação e movimento . . . 13

7.4 Alto-falantes virtuais × HRTF direta por bin . . . 14

7.5 Sala, externalização e head tracking . . . 14

7.6 Cancelamento de crosstalk (alto-falantes do celular) . . . 14

8. O coração do Tarkas: cálculo por frequência 15

8.1 Parâmetros da STFT . . . 15

8.2 Estatísticas por bin . . . 15

8.3 Separação direto / ambiente . . . 15

8.4 Onde cada som vai: as regras de destino . . . 16

8.5 Demonstração com dados reais . . . 16
8.6 Síntese binaural por bin sem artefatos . . . 17
8.7 Entrada multicanal (5.1, 7.1, 7.1.4) . . . 17
9. Custo computacional e latência **18**
9.1 Contagem de operações (estéreo → binaural, 48 kHz) . . . 18
9.2 Latência . . . 18
9.3 Regras de tempo real (já presentes no documento AUDIO_ENGINE do Tarkas) . . . 18
10. Arquitetura proposta no Tarkas **20**
10.1 Onde o motor entra . . . 20
10.2 Implementação: Kotlin ou C++? . . . 20
10.3 Esqueleto do código . . . 21
10.4 Integração com o que o Tarkas já tem . . . 21
10.5 Visualizador "onde cada som está" . . . 22
10.6 Parâmetros expostos ao usuário . . . 22
11. Loudness, testes e validação **23**
11.1 Loudness . . . 23
11.2 Testes objetivos (automatizáveis em JUnit, como o EqDspTest atual) . . . 23
11.3 Testes subjetivos . . . 23
12. Aspectos legais e de licenciamento **24**
13. Plano de implementação em fases **25**
14. Folha de fórmulas **26**
15. Referências **27**

1. Resumo executivo

O Dolby Atmos não é uma "caixa mágica" que descobre sozinho onde cada som deve ir. Ele funciona porque o **engenheiro de mixagem já decidiu** a posição de cada som em estúdio e gravou essas posições como **metadados** (coordenadas x, y, z, tamanho e alguns parâmetros extras) ao lado do áudio. Na reprodução, um **renderizador** lê esses metadados e calcula, em tempo real, quanto de cada som vai para cada alto-falante real, ou, no caso de fones, aplica filtros HRTF que enganam o cérebro para ouvir o som naquela direção.

O app Tarkas, porém, toca quase sempre **estéreo** (arquivos locais, YouTube Music, Spotify). Estéreo não tem metadado de posição. Por isso, o que o Tarkas precisa é de duas peças:

- **Analisador espacial por frequência (upmix):** a cada ~10,7 ms, o áudio é dividido em 1.025 faixas de frequência (STFT). Em cada faixa calculamos a diferença de nível e a coerência entre esquerda e direita. Isso diz *onde* aquela frequência foi colocada no mix original (pan) e se ela é som direto (voz, instrumento) ou ambiente (reverb, plateia). Essa é a resposta técnica para "saber onde cada som tem que ir".
- **Renderizador espacial:** cada faixa de frequência recebe uma direção 3D (azimute, elevação, distância) e é renderizada com HRTF (fones) ou com VBAP (alto-falantes/saída multicanal). Para conteúdo que já vem multicanal (5.1, 7.1, 7.1.4), pulamos a análise e posicionamos alto-falantes virtuais.

A análise mostra que o custo é baixo: cerca de **38 MFLOP/s** para estéreo → binaural a 48 kHz, o que ocupa por volta de 1 a 2% de um núcleo de um celular atual, com latência de aproximadamente 21 ms. O motor cabe no pipeline que o Tarkas já tem (Media3 `AudioProcessor`, o mesmo ponto onde hoje roda o `ParametricEqAudioProcessor`) e combina bem com o AutoEq que o app já possui: o AutoEq corrige o fone, o motor espacial cria a cena.

Pergunta	Resposta curta
O que é Atmos?	Áudio baseado em objetos: até 128 entradas (bed 7.1.2 + 118 objetos) com posição 3D em metadados, renderizadas para qualquer sala ou fone.
Dá para decodificar Atmos no Tarkas?	Só por decodificadores licenciados do aparelho (MediaCodec). Os objetos JOC normalmente não chegam ao app; recebemos 5.1/7.1 PCM no máximo.
Dá para fazer algo equivalente?	Sim, um motor próprio: upmix espectral + renderização HRTF/VBAP. Não pode se chamar Atmos.
"Cálculo por frequência"?	STFT de 2048 pontos, hop 512: 93,75 quadros/s × 1.025 bins = ~96 mil decisões de posição por segundo por canal.
Custo / latência	≈ 38 MFLOP/s, ≈ 21 ms de latência algorítmica, sem alocação no callback.

2. O que é o Dolby Atmos: história e conceito

2.1 Linha do tempo

Ano	Marco
2012	Estreia no cinema com *Valente* (Pixar). Sistema de cinema com até 64 saídas de alto-falante e até 128 entradas de áudio.
2014	Chegada ao home theater via Blu-ray (Dolby TrueHD com extensão Atmos) e alto-falantes "up-firing" com reflexão no teto.
2015–2017	Dolby Digital Plus (E-AC-3) com Joint Object Coding (JOC) para streaming; padronizado como ETSI TS 103 420. Lançamento do AC-4 (ETSI TS 103 190).
2019	Dolby Atmos Music: Amazon Music HD e Tidal passam a distribuir músicas em Atmos.
2021	Apple Music adota Atmos com o nome "Áudio Espacial", entregando E-AC-3 JOC e renderização binaural própria em fones.
2022–2024	Android 13 introduz a API `Spatializer` e o sensor de head tracking. AOM publica o IAMF (formato aberto, "Eclipsa Audio"), concorrente livre de royalties.

2.2 Três formas de representar som 3D

Todo sistema imersivo usa uma (ou uma mistura) de três representações. Entender a diferença é essencial para projetar o motor do Tarkas:

Representação	Como funciona	Exemplos	Vantagem / limite
Baseada em canais	Cada canal é o sinal de um alto-falante numa posição fixa.	Estéreo, 5.1, 7.1, 7.1.4	Simples; mas só toca certo no layout para o qual foi mixado.
Baseada em objetos	Cada som é um sinal mono + metadados de posição que mudam no tempo. O renderizador calcula os ganhos na hora.	Dolby Atmos, DTS:X, MPEG-H, IAMF	Independente do layout; exige renderizador e metadados.
Baseada em cena	O campo sonoro inteiro é decomposto em harmônicos esféricos (Ambisonics). Ordem N usa $(N+1)^2$ canais.	Ambisonics FOA/HOA, YouTube 360, Resonance Audio	Rotação barata (ótima para head tracking); resolução limitada pela ordem.

O Atmos é **híbrido**: usa um **bed** (camada de canais, normalmente 7.1.2) para sons difusos e **objetos** para sons que precisam de posição precisa ou de movimento.

3. Arquitetura do Dolby Atmos

3.1 Beds, objetos e as 128 entradas

Na produção, o renderizador da Dolby recebe até **128 entradas**. Na prática de música e home entertainment, as 10 primeiras costumam formar o bed 7.1.2 (L, R, C, LFE, Lss, Rss, Lrs, Rrs, Lts, Rts) e as **118 restantes** são objetos. A documentação do Cubase/Nuendo descreve exatamente esse limite: um bed e até 118 objetos no modo de objetos do MultiPanner. O bed é um "pré-mix" com o pan já embutido; o objeto é um sinal mono (ou par estéreo como dois objetos) com metadados dinâmicos.

3.2 Metadados de um objeto

Campo	Significado	Uso no renderizador
Posição x, y, z	Coordenadas numa sala normalizada (cubo). No ADM cartesiano, x e y vão de −1 a +1 e z de −1 a +1 (0 = altura do ouvido, +1 = teto).	Define para quais alto-falantes o som vai e com que ganho.
Size (tamanho)	Quão grande/espalhado o objeto soa (0 = ponto).	Espalha a energia em mais alto-falantes; em binaural aumenta a descorrelação.
Snap	Força o objeto a tocar no alto-falante mais próximo.	Evita imagem fantasma (útil para diálogo).
Zone mask	Exclui zonas da sala (ex.: "sem alto-falantes de trás").	Recalcula o pan só com os alto-falantes permitidos.
Binaural render mode	Off / Near / Mid / Far.	Só em fones: muda a distância percebida e a quantidade de "sala" (reverb) aplicada.
Ganho / trim / downmix	Ajustes por layout de reprodução.	Mantém o equilíbrio ao tocar em 5.1, 2.0 etc.

Segundo a documentação do Logic Pro, cada bed e objeto pode receber Off, Near, Mid ou Far, com Mid como padrão. Guias de mixagem relatam que Near soa quase anecoico e Far tem mais reverberação. Isso mostra um ponto importante para o Tarkas: a sensação de "fora da cabeça" vem em grande parte da **sala simulada**, não só da direção.

3.3 Do estúdio ao seu ouvido

- **Autoria:** a DAW (Pro Tools, Logic, Nuendo) envia bed + objetos + metadados para o Dolby Atmos Renderer (antes, uma máquina dedicada chamada RMU).
- **Master:** salvo como *Dolby Atmos Master File* (.atmos) ou como **ADM BWF**: um arquivo WAV Broadcast Wave com metadados no Audio Definition Model (norma ITU-R BS.2076). O ADM é aberto e é o formato de intercâmbio entre ferramentas.
- **Cinema:** empacotado como IAB (Immersive Audio Bitstream, SMPTE ST 2098-2).
- **Distribuição doméstica/streaming:** o encoder faz **spatial coding**: agrupa os 118 objetos em um número menor de "elementos" dinâmicos (tipicamente da ordem de 12 a 16) para caber no bitrate. Depois codifica em TrueHD (Blu-ray), E-AC-3 JOC (streaming, ex.: 768 kbps na Apple Music) ou AC-4.
- **Reprodução:** o decodificador reconstrói os elementos e o renderizador do aparelho (receiver, TV, soundbar, celular) gera as saídas para o layout real ou o sinal binaural para fones.

3.4 Os codecs

Codec	Norma	Como carrega o Atmos
Dolby TrueHD	Proprietário (sem perdas)	Núcleo 7.1 + subfluxo de extensão com objetos.
E-AC-3 JOC (Dolby Digital Plus Atmos)	ETSI TS 102 366 + ETSI TS 103 420	Envia um downmix de canais (ex.: 5.1) compatível com qualquer decodificador DD+. Parâmetros JOC no container EMDF permitem reconstruir os objetos a partir do downmix. Quem não entende JOC toca só o 5.1.
AC-4	ETSI TS 103 190-1/-2	Codec de nova geração com objetos nativos e modo IMS (Immersive Stereo), um estéreo pré-renderizado otimizado para celulares e fones.
ADM BWF	ITU-R BS.2076 / BS.2088	Formato de master, não de streaming. Aberto.

Joint Object Coding, em termos matemáticos, é uma reconstrução paramétrica: o encoder envia o downmix D (M canais) e, por banda de frequência e por quadro de tempo, uma matriz de coeficientes C. O decodificador estima os N objetos como $\hat{O} = C \cdot D$ (com a adição de sinais descorrelacionados para recuperar a largura). É a mesma família de técnicas do MPEG Spatial Audio Object Coding. Ou seja: até o Atmos trabalha **por banda de frequência**, exatamente o tipo de cálculo que o Tarkas fará.

4. Coordenadas e layouts de alto-falantes

4.1 Dois sistemas de coordenadas

Existem duas maneiras de descrever a posição de um som, e o motor precisa converter entre elas:

- **Alocêntrica (sala, cartesiana):** "o som está no canto de trás à esquerda, no teto". É o modelo da Dolby: a posição é relativa às paredes, então numa sala retangular o som fica colado na parede, qualquer que seja o tamanho da sala.
- **Egocêntrica (ouvinte, polar):** "o som está a 135° de azimute e 45° de elevação". É o que HRTF e VBAP precisam.

Conversão cartesiano → polar (convenção ADM: x para a direita, y para a frente, z para cima; azimute positivo à esquerda, como no ITU-R BS.2051):

$$az = \text{atan2}(-x, y) \cdot el = \text{atan2}(z, \sqrt{(x^2 + y^2)}) \cdot r = \sqrt{(x^2 + y^2 + z^2)}$$

$$x = -r \cdot \text{sen}(az) \cdot \cos(el) \cdot y = r \cdot \cos(az) \cdot \cos(el) \cdot z = r \cdot \text{sen}(el)$$

Cuidado: posições do cubo alocêntrico não são ângulos. O ponto (x = −1, y = 1) no cubo é "canto frontal esquerdo", que no layout 7.1.4 corresponde ao alto-falante L em +30°, e não a +45°. A norma ITU-R BS.2127 (renderizador ADM de referência da EBU, código aberto em *libear*) define um mapeamento por partes entre o cubo e os ângulos reais dos alto-falantes. Use esse mapeamento, não o atan2 puro, ao interpretar posições vindas de arquivos ADM.

4.2 Layouts padronizados (ITU-R BS.2051)

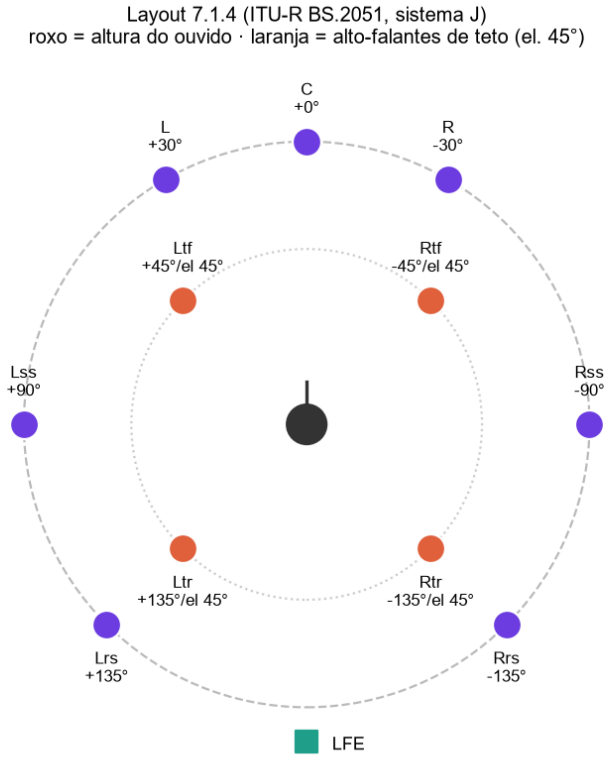


Figura 1 – Posições nominais do layout 7.1.4 (vista de cima). Os ângulos de azimute são positivos à esquerda. São estes os "alto-falantes virtuais" do modo binaural do Tarkas.

Canal	Azimute	Elevação	Faixa permitida (BS.2051)
L / R	+30° / −30°	0°	±30° (até ±45° em alguns sistemas)
C	0°	0°	—

Canal	Azimute	Elevação	Faixa permitida (BS.2051)
LFE	—	—	Sem direção (< 120 Hz)
Lss / Rss (laterais)	±90°	0°	±60° a ±110°
Lrs / Rrs (traseiros)	±135°	0°	±120° a ±150°
Ltf / Rtf (teto frontal)	±45°	+45°	±30° a ±45°; elevação 30° a 55°
Ltr / Rtr (teto traseiro)	±135°	+45°	±110° a ±135°; elevação 30° a 55°

5. Psicoacústica: como o cérebro localiza sons

Todo o "truque" do áudio espacial é reproduzir no tímpano as pistas que o cérebro usa para localizar sons. São quatro famílias de pistas, e cada uma trabalha numa faixa de frequência diferente. Por isso o cálculo **precisa** ser por frequência.

5.1 ITD – diferença de tempo entre os ouvidos

Um som à direita chega antes ao ouvido direito. O modelo clássico de Woodworth, para uma cabeça esférica de raio *a* e velocidade do som *c*, é:

$$\mathrm{ITD}(\theta) = (a / c) \cdot (\theta + \operatorname{sen}\theta), \text{ com } \theta \text{ em radianos, } 0 \le \theta \le \pi/2$$

Cálculo: com a = 0,0875 m e c = 343 m/s, a ITD máxima (θ = 90°) é 0,0875/343 × (1,5708 + 1) = **0,656 ms**, o que equivale a **31,5 amostras a 48 kHz**. Esse é o valor de alta frequência; abaixo de ~1 kHz a ITD real é maior, cerca de 3a/c·sen θ ≈ 0,765 ms (Kuhn, 1977), e o modelo de Brown & Duda do protótipo mede 0,78 ms. O cérebro detecta diferenças de cerca de 10 µs (menos de meia amostra!), por isso os atrasos precisam de **interpolação fracionária** (filtro all-pass ou Lagrange de 3ª ordem), e não de atraso inteiro.

A ITD de fase só é útil abaixo de ~1,5 kHz: acima disso, o comprimento de onda (c/f = 343/1500 ≈ 23 cm) fica menor que o caminho entre os ouvidos e a fase fica ambígua. Acima dessa frequência o cérebro usa a ITD do *envelope* (ataques, transientes).

5.2 ILD – diferença de nível entre os ouvidos

A cabeça faz sombra para frequências cujo comprimento de onda é menor que ela. Em 200 Hz a ILD é quase zero; em 6 kHz pode passar de 15–20 dB. É a **Teoria Duplex** de Lord Rayleigh (1907): ITD domina os graves, ILD domina os agudos. Brown e Duda (1998) propuseram um modelo estrutural barato: um filtro de 1 polo e 1 zero para a sombra de cabeça, mais atraso de ITD, mais ecos curtos da orelha.

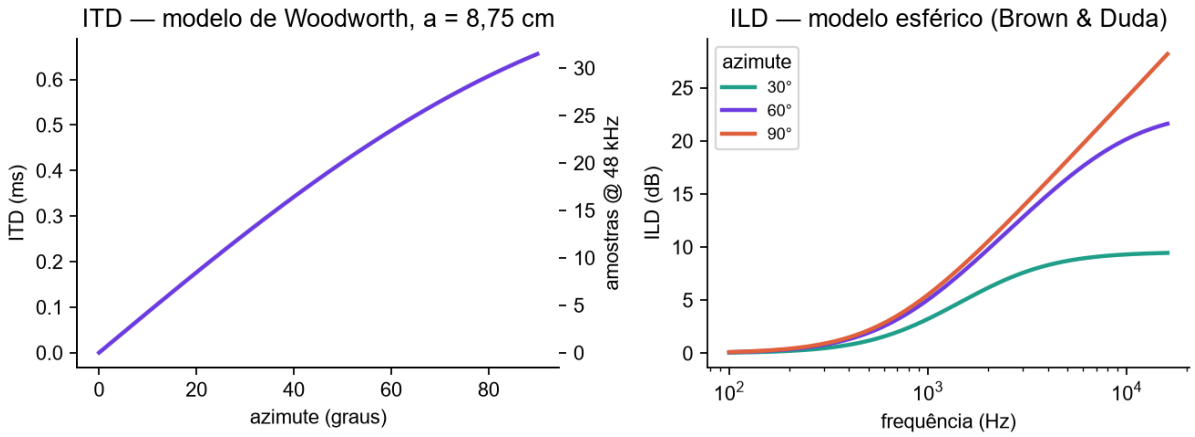


Figura 2 – Esquerda: ITD pelo modelo de Woodworth (máx. 0,656 ms ≈ 31,5 amostras a 48 kHz). Direita: ILD do modelo esférico de Brown & Duda; a diferença cresce com a frequência e com o azimute.

5.3 Pistas espectrais da orelha (pinna) e elevação

ITD e ILD não distinguem frente de trás nem cima de baixo: todos os pontos de um "cone de confusão" em torno do eixo dos ouvidos têm ITD e ILD praticamente iguais. Quem resolve é a **orelha externa**, que cria picos e vales (notches) entre ~4 e 16 kHz. O notch principal, perto de 6–10 kHz, sobe de frequência conforme a fonte sobe. Essas pistas são **individuais**, pois cada orelha é diferente, e essa é a maior fonte de erro em binaural genérico: confusão frente/trás e sons "dentro da cabeça".

Blauert descreveu as **bandas direcionais**: o cérebro tende a ouvir bandas estreitas em torno de 8 kHz como "acima" e bandas em torno de 300 Hz e 3 kHz como "à frente". Junto com o **efeito Pratt** (sons mais

agudos são percebidos como mais altos no espaço), isso dá base científica para a heurística de elevação por frequência da seção 8.

5.4 Distância, sala e precedência

- **Lei do inverso do quadrado:** nível direto cai 6 dB a cada dobro de distância: $L(r) = L(r_0) - 20 \cdot \log_{10}(r/r_0)$.
- **Razão direto/reverberante (DRR):** a pista de distância mais forte em salas. O nível reverberante é quase constante; o direto cai. Por isso os modos Near/Mid/Far do Atmos mudam a quantidade de sala.
- **Absorção do ar:** agudos caem com a distância (≈ alguns dB por 100 m em 10 kHz, que só importa para efeitos de "longe").
- **Efeito de precedência (Haas):** com dois sinais iguais separados por 1 a ~30 ms, ouvimos a direção do primeiro. Isso serve para posicionar sem mudar o nível, mas também cria artefatos se o upmix atrasar canais sem cuidado.
- **Ângulo mínimo audível:** cerca de 1° à frente, 5° a 10° nas laterais. Precisão alta na frente, baixa dos lados: a grade de HRTF pode ser mais densa à frente.

5.5 Resolução de frequência do ouvido

A cóclea funciona como um banco de filtros com largura de banda crescente. A largura de banda retangular equivalente (Glasberg & Moore, 1990) é:

$$\mathrm{ERB}(f) = 24{,}7 \cdot (4{,}37 \cdot f/1000 + 1)\ \mathrm{Hz} \cdot \text{ERB-rate}(f) = 21{,}4 \cdot \log_{10}(1 + 0{,}00437 \cdot f)$$

Em 100 Hz, ERB ≈ 35 Hz; em 1 kHz, ≈ 133 Hz; em 10 kHz, ≈ 1,1 kHz. O espectro audível cabe em ~40 bandas ERB. Consequência prática: **não precisamos de 1.025 decisões independentes**. Calculamos as estatísticas em cada bin, mas suavizamos os ganhos em ~40 bandas ERB. Isso reduz "ruído musical" (artefato de bins piscando) e custo.

6. A matemática do panning

6.1 Leis de pan entre dois alto-falantes

A lei **seno/cosseno** (potência constante, −3 dB no centro) é a base: para pan p de −1 (esquerda) a +1 (direita):

$$g_L = \cos((p+1)\cdot\pi/4) \cdot g_R = \text{sen}((p+1)\cdot\pi/4) \rightarrow g_L^{\,2} + g_R^{\,2} = 1$$

A **lei das tangentes** relaciona os ganhos ao ângulo percebido θ entre alto-falantes em $\pm\theta_0$: $\tan\theta / \tan\theta_0 = (g_L - g_R) / (g_L + g_R)$. É ela que usamos ao contrário no upmix: medimos g_L e g_R em cada bin e obtemos o ângulo.

6.2 VBAP – Vector Base Amplitude Panning (Pulkki, 1997)

O VBAP generaliza o pan para qualquer layout 3D. Os alto-falantes são divididos em triângulos (triangulação de Delaunay da casca convexa sobre a esfera). Para uma fonte na direção unitária **p** e o triângulo com alto-falantes $\mathbf{l}_1, \mathbf{l}_2, \mathbf{l}_3$ (vetores unitários nas linhas da matriz **L**):

$$\mathbf{g} = \mathbf{p}^T \cdot \mathbf{L}^{-1} \quad \mathbf{g}_{norm} = \mathbf{g} / \|\mathbf{g}\|$$

Escolhe-se o triângulo em que todos os ganhos são ≥ 0. $\mathbf{L}^{-1}$ é pré-calculada para cada triângulo na inicialização, então cada atualização de posição custa só uma multiplicação 3×3 (9 multiplicações). Para objetos com **size**, usa-se MDAP (Multiple-Direction Amplitude Panning): soma-se o VBAP de várias direções espalhadas em torno de **p** e normaliza-se a potência.

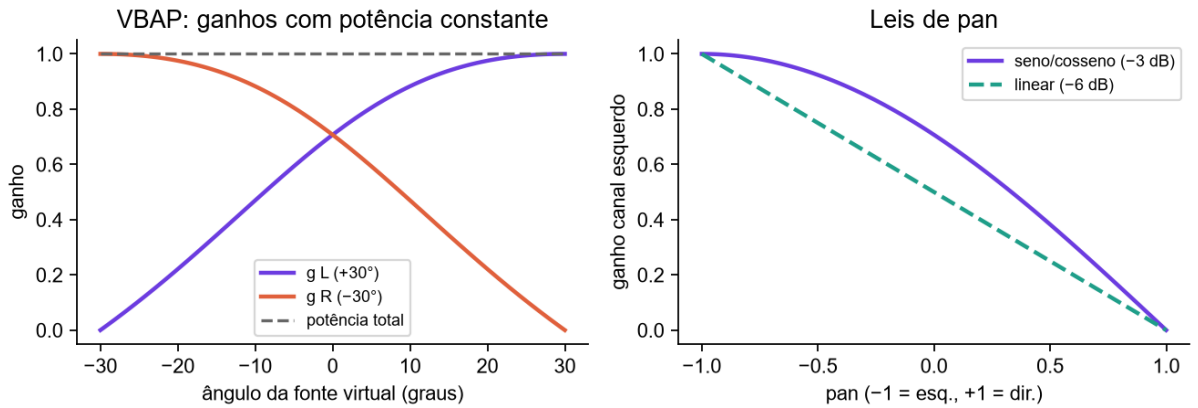


Figura 3 – Esquerda: ganhos VBAP entre L(+30°) e R(−30°); a potência total é constante, então o volume não "afunda" no meio. Direita: comparação entre leis de pan.

6.3 O panner do Atmos

O renderizador Atmos usa um panner **alocêntrico** próprio, não VBAP. A especificação exata não é pública. O que é público é o comportamento: posições num cubo, sons colados nas paredes e nos cantos, separação em "camadas" (ouvido e teto). A norma ITU-R BS.2127 inclui um panner alocêntrico cartesiano compatível com esse modelo, além do panner de fonte pontual baseado em VBAP. Para o Tarkas recomendamos **VBAP sobre o layout virtual 7.1.4** (simples, bem documentado, sem patente) e o mapeamento cubo→ângulo da BS.2127 apenas se algum dia lermos arquivos ADM.

6.4 Gestão de graves

Abaixo de ~120 Hz não há localização útil em sala (comprimento de onda > 2,8 m), e mover graves entre canais virtuais em binaural causa "embolamento" e variação de volume. Regra: separar com um crossover Linkwitz-Riley de 4ª ordem em 120 Hz (soma plana em fase) e manter os graves **centralizados e coerentes**.

7. Renderização binaural (fones)

7.1 HRTF e HRIR

A **HRTF** (Head-Related Transfer Function) é a resposta em frequência do caminho entre uma direção no espaço e o tímpano, medida com microfones dentro dos ouvidos. No tempo ela se chama **HRIR** (tipicamente 256 a 512 amostras a 48 kHz). Renderizar uma fonte *s* numa direção (θ, φ) é uma convolução por ouvido:

$$y_L(t) = (h_{L,\theta,\varphi} * s)(t) \cdot y_R(t) = (h_{R,\theta,\varphi} * s)(t) \text{ ou, na frequência: } Y_L(k) = H_{L,\theta,\varphi}(k) \cdot S(k)$$

A forma de frequência (à direita) é o ponto-chave: no domínio da frequência, posicionar um som é **multiplicar cada bin por um número complexo**. Se cada bin tiver sua própria direção, multiplicamos cada bin pela HRTF da sua direção. É isso que permite o cálculo "por frequência" pedido para o Tarkas.

7.2 Bases de HRTF abertas (formato SOFA, AES69)

Base	Conteúdo	Licença (verificar antes de distribuir)
SADIE II (Univ. de York)	Manequins KU100 e KEMAR e 18 humanos; WAV e SOFA a 44,1/48/96 kHz.	A página antiga indicava Apache 2.0; a atual não nomeia a licença com clareza. Conferir o campo de licença no SOFA ou com a universidade.
MIT KEMAR (Gardner & Martin, 1994)	Manequim KEMAR, 710 direções, 44,1 kHz.	Uso livre com atribuição; muito usado.
CIPIC (UC Davis)	45 sujeitos, 1.250 direções, com medidas antropométricas.	Uso livre para pesquisa; verificar uso comercial.
ARI (Áustria)	Mais de 200 sujeitos, alta densidade.	CC BY-SA.
HUTUBS (TU Berlin)	96 sujeitos, HRTFs medidas e simuladas.	CC BY.

Leitura em C/C++: **libmysofa** (BSD) lê SOFA, faz busca da direção mais próxima e interpolação. Para celular, recomendamos **pré-converter** no build: reamostrar para 48 kHz, cortar para 256 amostras, extrair a ITD e transformar o restante em **fase mínima**, depois salvar num binário próprio pequeno (assets).

7.3 Interpolação e movimento

Bases têm direções discretas. Para direções intermediárias:

- **Separar ITD da magnitude:** HRIR de fase mínima + atraso puro. Interpolar fases cruas cria cancelamentos (efeito pente).
- **Interpolação baricêntrica** nos triângulos da grade (os mesmos pesos do VBAP): $H = w_1H_1 + w_2H_2 + w_3H_3$ nas magnitudes; $ITD = w_1\tau_1 + w_2\tau_2 + w_3\tau_3$.
- **Pré-computar uma grade densa** (ex.: 5° em azimute × 10° em elevação ≈ 1.300 direções) e no tempo real usar a mais próxima, ou bilinear. Memória: 1.300 × 2 ouvidos × 1.025 bins × 4 bytes (só magnitude) ≈ 10,7 MB; com magnitude em 16 bits e 513 bins ≈ 2,7 MB.
- **Crossfade de ganhos** entre quadros para direções que mudam rápido (suavização exponencial com $\tau \approx$ 20–50 ms).

7.4 Alto-falantes virtuais × HRTF direta por bin

Abordagem	Como	Custo (48 kHz)	Quando usar
Alto-falantes virtuais	Renderiza a cena num layout 7.1.4 (VBAP) e convolve cada alto-falante com a HRIR fixa dele.	11 canais × 2 ouvidos = 22 convoluções de 256 taps (FFT particionada): ≈ 60–90 MFLOP/s	Conteúdo multicanal (5.1/7.1/7.1.4) e objetos.
HRTF direta por bin	Cada bin da STFT recebe uma direção e é multiplicado pela HRTF dela.	≈ 10 MFLOP/s além da STFT	Upmix de estéreo (caso principal do Tarkas).
Ambisonics (3ª ordem)	Codifica tudo em 16 canais, gira a cena, decodifica binaural (MagLS).	16 × 2 convoluções + rotação barata	Head tracking com muitos objetos.

7.5 Sala, externalização e head tracking

HRTF anecoica pura soa "dentro da cabeça". Externalização exige: (1) reflexões iniciais (5–80 ms) com direções coerentes, por exemplo com um modelo de fonte-imagem (image source) de uma sala pequena; (2) cauda reverberante difusa e descorrelacionada entre os ouvidos (Feedback Delay Network de 8 ou 16 linhas, RT60 de 0,3 a 0,6 s); (3) head tracking. Os presets do Tarkas ("Estúdio", "Sala", "Arena") equivalem aos Near/Mid/Far.

Head tracking: no Android 13+, fones compatíveis expõem `Sensor.TYPE_HEAD_TRACKER` (rotação em vetor de eixo-ângulo). O motor converte para quaternion $\mathbf{q}$ e gira cada direção de fonte com $\mathbf{q}^{-1}$, mantendo a cena parada enquanto a cabeça gira. A documentação do AOSP pede latência de head tracking de no máximo 150 ms; o ideal perceptivo fica abaixo de ~60 ms. Atualizar as direções a cada quadro de STFT (10,7 ms) é suficiente.

7.6 Cancelamento de crosstalk (alto-falantes do celular)

Em alto-falantes estéreo do celular, cada ouvido escuta os dois alto-falantes. O cancelamento de crosstalk (CTC) inverte a matriz 2×2 de caminhos acústicos por frequência: $\mathbf{C}(k) = (\mathbf{H}(k) + \beta\mathbf{I})^{-1}$, com regularização β para não explodir em frequências mal-condicionadas. Com os alto-falantes a ~15 cm um do outro, o efeito é limitado e depende muito da posição da cabeça. Recomendação: um modo "alargar" leve, não binaural completo.

8. O coração do Tarkas: cálculo por frequência

Esta é a parte que responde diretamente ao pedido: **para cada frequência tocada, calcular em tempo real onde ela está e para onde deve ir**. O método é baseado na linha de pesquisa de Avendano e Jot (ICASSP 2002 e JAES 2004, *A Frequency-Domain Approach to Multichannel Upmix*), que compara as STFTs dos canais esquerdo e direito para estimar o pan de cada componente e separar ambiência via coerência, mais a extensão de renderizar cada bin diretamente com HRTF.

8.1 Parâmetros da STFT

Parâmetro	Valor	Por quê
Taxa de amostragem	48 kHz (reamostrar 44,1 kHz se preciso, ou ter HRTFs nas duas taxas)	Taxa nativa da maioria dos DACs Android.
Janela de análise L	1.024 amostras, Hann	21,3 ms; resolução ~47 Hz, separa notas no grave médio.
Tamanho da FFT N	2.048 (zero-padding de 1.024)	Espaço para a HRIR sem aliasing circular (ver 8.6).
Hop H	512 (50% de sobreposição)	Hann com 50% soma constante: reconstrução perfeita com OLA.
Quadros por segundo	48.000 / 512 = 93,75	Uma decisão de posição a cada 10,7 ms por bin.
Bins	N/2 + 1 = 1.025, espaçamento 23,4 Hz	~96.000 decisões de posição por segundo.

8.2 Estatísticas por bin

Para cada quadro m e bin k, com X_L e X_R as STFTs, mantemos médias exponenciais (α ≈ 0,7–0,9, o que equivale a ~30–100 ms de memória):

$$\Phi_{LL} \leftarrow \alpha\cdot\Phi_{LL} + (1-\alpha)\cdot|X_L|^2 \cdot \Phi_{RR} \leftarrow \alpha\cdot\Phi_{RR} + (1-\alpha)\cdot|X_R|^2 \cdot \Phi_{LR} \leftarrow \alpha\cdot\Phi_{LR} + (1-\alpha)\cdot X_L\cdot X_R^{*}$$

Coerência (quanto L e R são o mesmo sinal, 1 = fonte direta pan-eada, ≈ 0 = ambiência/reverb):

$$\psi(k) = |\Phi_{LR}| \,/\, \sqrt{(\Phi_{LL}\cdot\Phi_{RR})}$$

Pan estimado (inverso da lei seno/cosseno, de −1 a +1):

$$p(k) = (4/\pi)\cdot \operatorname{atan2}(\sqrt{\Phi_{RR}}, \sqrt{\Phi_{LL}}) - 1$$

Azimute no palco original (lei das tangentes inversa, $\theta_0 = 30°$) e **alargamento** controlado pelo usuário (fator w ≥ 1, até ±110° para envolver):

$$\theta_{orig}(k) = \operatorname{atan}\left(\tan\theta_0 \cdot \frac{\sqrt{\Phi_{LL}} - \sqrt{\Phi_{RR}}}{\sqrt{\Phi_{LL}} + \sqrt{\Phi_{RR}}}\right) \quad \theta(k) = \operatorname{clamp}(w\cdot\theta_{orig}, -110°, +110°)$$

8.3 Separação direto / ambiente

Modelo: $X_L = g_L\cdot S + A_L$, $X_R = g_R\cdot S + A_R$, com A_L e A_R descorrelacionados e de mesma potência. Uma máscara simples e robusta usa a coerência:

$$D(k) = \max(\psi(k), \lambda(k))^{\gamma} \quad (\gamma \approx 2) \quad \lambda = \frac{|\Phi_{LL} - \Phi_{RR}|}{(\Phi_{LL} + \Phi_{RR})}$$

$$\text{direto} = \sqrt{D}\cdot X \quad \text{ambiente} = \sqrt{(1-D)}\cdot X$$

Correção do protótipo (fase 0): com uma fonte toda num lado só (R = 0), a coerência vira 0/0 e o bin seria tratado como ambiente. A dominância de nível λ resolve: um som só na esquerda tem λ = 1, logo D = 1.

A forma exata (decomposição PCA por bin) também é barata: o autovetor dominante da matriz 2×2 $[[\Phi_{LL}, \Phi_{LR}], [\Phi_{LR}^{*}, \Phi_{RR}]]$ dá a direção do primário e o menor autovalor estima a potência ambiente.

Recomendação: começar com ψ^{γ} e, se testes de escuta mostrarem vazamento, migrar para PCA.

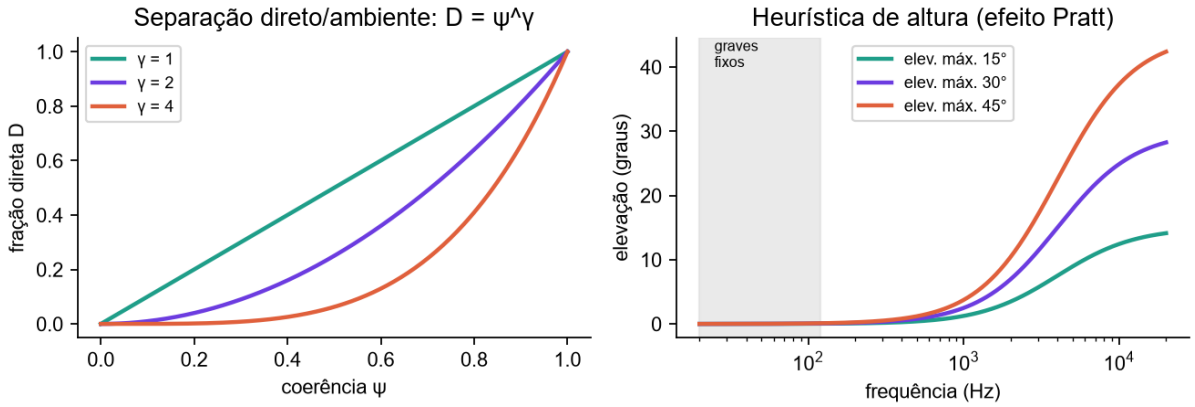


Figura 4 – Esquerda: fração direta em função da coerência. Direita: heurística de elevação por frequência (curva logística em oitavas), com os graves travados no plano do ouvido.

8.4 Onde cada som vai: as regras de destino

Componente	Destino (az, el)	Justificativa
Graves < 120 Hz	Centro, el. 0°, sem descorrelação	Sem localização útil; evita embolamento e variação de nível.
Direto, coerência alta	θ(k) alargado; el(f) = e_{max} / (1 + exp(−s·log2(f/f_0))), $f_0 \approx 4$ kHz	Mantém o palco do mix; leve elevação dos agudos (efeito Pratt, bandas de Blauert).
Direto no centro (|p| < 0,1), 300 Hz–4 kHz	0° frontal, el. 0°, "âncora" sem alargamento	Voz principal fica presa à frente, como o canal C do Atmos.
Ambiente	Dividido entre laterais (±90°), traseiros (±135°) e teto (±45°/+45° e ±135°/+45°), com descorrelação	Envolvimento; é o papel do bed e dos alto-falantes de teto.
Transientes fortes (fluxo espectral alto)	Reduz o alargamento por 20–50 ms	Preserva o ataque e evita "espalhar" a bateria.

Descorrelação do ambiente: cada direção de ambiente recebe uma rotação de fase pseudoaleatória fixa por bin, $e^{j\varphi_d(k)}$, suavizada em frequência para não criar transientes difusos. Isso custa uma multiplicação complexa por bin por direção e cria a sensação de "sala em volta".

8.5 Demonstração com dados reais

Para validar o método, geramos um mix estéreo sintético com três fontes pan-eadas com valores conhecidos e ruído ambiente descorrelacionado, e rodamos exatamente o algoritmo acima (Python/NumPy, mesmos parâmetros da tabela 8.1). Resultados medidos nos bins das fontes:

Fonte	Pan real	Pan estimado	Coerência ψ
Baixo 110 Hz	0,00	−0,001	1,000
Voz/sint. 440 Hz (+ harmônico 880 Hz)	−0,70	−0,699 / −0,703	1,000
Guitarra 2,5 kHz	+0,60	+0,601	1,000
Ambiência (5–15 kHz, ruído descorrelacionado)	—	espalhado	0,31 (média)

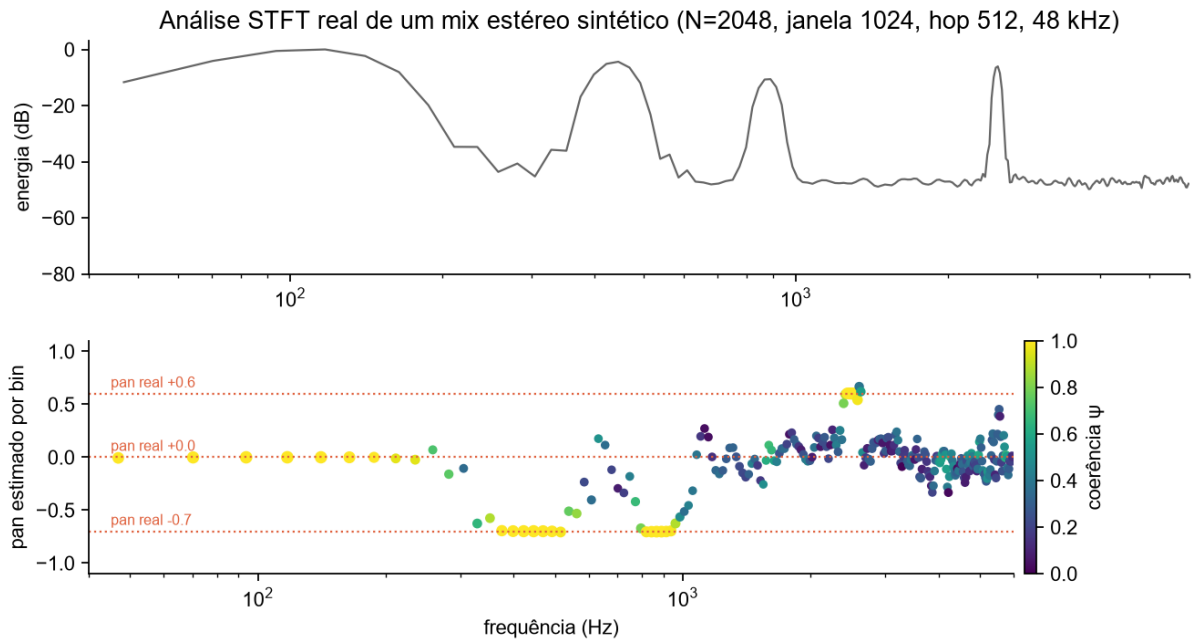


Figura 5 – Saída real do analisador. Cada ponto é um bin; a cor é a coerência. As fontes aparecem exatamente nos pans corretos com ψ ≈ 1; o ruído ambiente aparece com coerência baixa e é enviado às direções de envolvimento.

8.6 Síntese binaural por bin sem artefatos

Multiplicar um quadro por uma HRTF equivale a uma convolução **circular** de tamanho N. Para que ela seja linear (sem "eco" do fim do quadro dobrando no começo), é preciso L + M − 1 ≤ N, onde L é a janela e M é o comprimento da HRIR. Com L = 1.024 e N = 2.048, a HRIR pode ter até M = 1.025 amostras. Usando HRIR de 256, sobra margem também para a suavização dos ganhos entre quadros. A síntese:

$$Y_L(k) = H_L(\theta_k, \varphi_k, k)\cdot\sqrt{D}\cdot X_M(k) + \Sigma_d\, H_L(d, k)\cdot e^{j\varphi_d(k)}\cdot a_d\cdot\sqrt{(1-D)}\cdot X_M(k)$$

Mistura seco/espacial (fase 0): somar o estéreo original e o sinal espacial cria filtro pente, porque as HRTFs atrasam e giram a fase. No protótipo isso abriu buracos de até −13 dB. A solução é interpolar magnitude e fase separadamente: Y = ((1−k)|D| + k|W|) · u, com u o fasor unitário de (1−k)·D/|D| + k·W/|W|. Com isso o pior desvio caiu para −7 dB, e esse resto é o notch da orelha em 7,3 kHz, uma pista de localização legítima.

onde X_M é a componente da fonte (o primário estimado, ou simplesmente $(X_L + X_R)/\sqrt{2}$ corrigido pelo pan) e a_d são os pesos das direções de ambiente ($\Sigma\, a_d^2 = 1$). Depois: IFFT de 2.048, overlap-add com hop 512, sem janela de síntese (a Hann de análise já soma constante).

Simplificação essencial: as HRTFs das direções de ambiente são fixas, então $\Sigma_d\, H_L(d,k)\cdot e^{j\varphi_d(k)}\cdot a_d$ pode ser **pré-computada** como um único filtro complexo por ouvido. O ambiente inteiro custa então só uma multiplicação complexa por bin por ouvido.

8.7 Entrada multicanal (5.1, 7.1, 7.1.4)

Se o decodificador entrega PCM multicanal (por exemplo E-AC-3 decodificado pelo MediaCodec do aparelho em 5.1), não há o que adivinhar: cada canal é um alto-falante virtual na posição da tabela 4.2. Renderizamos cada canal com a HRTF fixa dele (convolução particionada uniforme, partições de 256) e somamos; o LFE é somado mono aos dois ouvidos com o ganho de +10 dB em banda previsto para o canal LFE (ajustável pelo usuário). Opcionalmente, o upmix espectral pode rodar entre pares de canais (ex.: L/Lss) para "preencher" o teto quando não há canais de altura.

9. Custo computacional e latência

9.1 Contagem de operações (estéreo → binaural, 48 kHz)

Usando a estimativa padrão de ~2,5·N·$\log_2$N FLOP para uma FFT real de N pontos (N = 2.048, $\log_2$N = 11 → ~56 mil FLOP por FFT):

Etapa	Por quadro	× 93,75 quadros/s
2 FFTs diretas (L, R)	≈ 113 k FLOP	≈ 10,6 MFLOP/s
2 FFTs inversas (ouvido E, D)	≈ 113 k FLOP	≈ 10,6 MFLOP/s
Estatísticas + coerência + pan + máscaras (≈ 60 FLOP × 1.025 bins)	≈ 62 k FLOP	≈ 5,8 MFLOP/s
HRTF direta + ambiente pré-somado (≈ 100 FLOP × 1.025 bins)	≈ 103 k FLOP	≈ 9,6 MFLOP/s
Conversões, suavização, limitador	≈ 16 k FLOP	≈ 1,5 MFLOP/s
Total	**≈ 407 k FLOP**	**≈ 38 MFLOP/s**

Um núcleo Cortex-A7x com NEON chega a vários GFLOP/s de pico em float32; mesmo assumindo eficiência real de 2–4 GFLOP/s, o motor usa **~1–2% de um núcleo**. Com bibliotecas otimizadas (PFFFT, KissFFT com NEON, ou Ne10), o gargalo nem é a FFT. Para o modo de alto-falantes virtuais 7.1.4, adicione ~60–90 MFLOP/s.

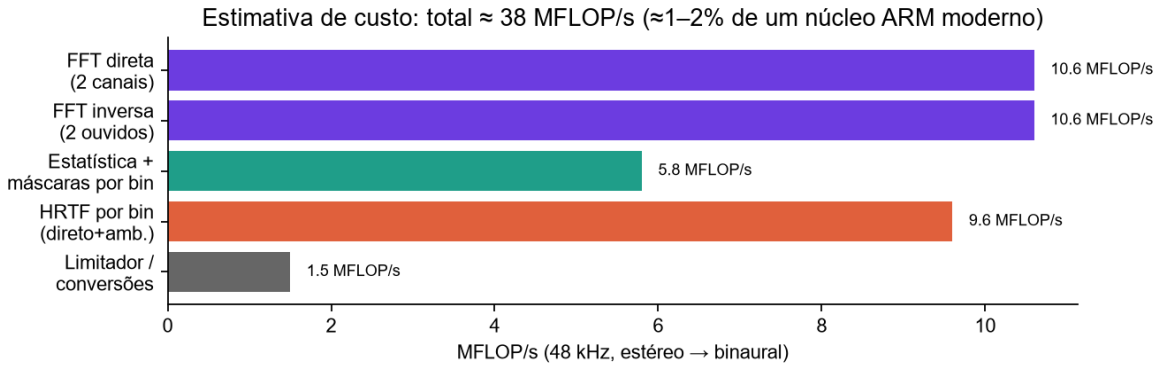


Figura 6 – Distribuição estimada do custo por etapa.

9.2 Latência

- **Algorítmica:** precisamos acumular uma janela de 1.024 amostras → 21,3 ms. Com hop de 512, a primeira saída sai após 1.024 amostras; depois, a cada 512.
- **Música:** irrelevante. **Vídeo (YouTube Music com clipe):** 21 ms é inferior ao limiar típico de percepção de dessincronia labial (~45 ms de atraso de áudio), mas deve ser somado à latência do Bluetooth. Reportar o atraso para o player compensar, ou usar janela de 512 no modo vídeo.
- **Head tracking:** latência total = sensor + Bluetooth + 10,7 ms de quadro. Fica bem abaixo dos 150 ms exigidos pelo AOSP.

9.3 Regras de tempo real (já presentes no documento AUDIO_ENGINE do Tarkas)

- Nenhuma alocação, lock, log ou JNI-callback dentro do processamento: todos os buffers são pré-alocados no configure.
- Parâmetros vindos da UI trocados por variáveis atômicas e suavizados no áudio (rampas de 20–50 ms).

- Troca de HRTF/preset por *double buffering*: carrega em outra thread, troca o ponteiro atomicamente, faz crossfade de um quadro.
- Proteções numéricas: ε nos denominadores, flush de denormais (FTZ/DAZ no ARM), NaN-guard que zera o estado e cai para bypass.

10. Arquitetura proposta no Tarkas

10.1 Onde o motor entra

Hoje o Tarkas já processa áudio no ExoPlayer por meio do `ParametricEqAudioProcessor` (Media3 `BaseAudioProcessor`) que chama o `EqDsp` em float. O motor espacial entra como **outro AudioProcessor na mesma cadeia**, com uma diferença: ele muda o número de canais (N canais de entrada → 2 de saída binaural, ou → 8/12 para saída multicanal real por HDMI/USB).

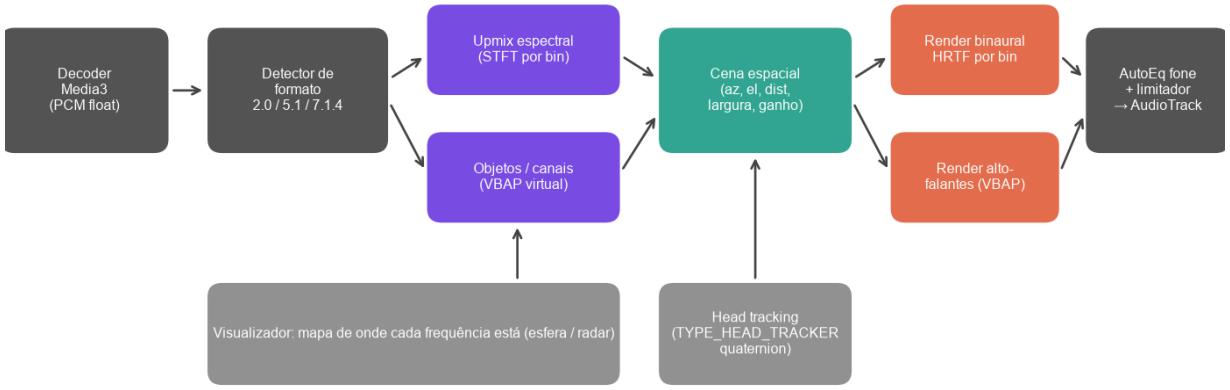


Figura 7 – Pipeline proposto. O AutoEq (correção do fone) fica depois do binaural, porque corrige o transdutor, e não a cena.

Ordem	Processador	Observação
1	SpatialAudioProcessor (novo)	Entrada 2.0/5.1/7.1 PCM float, saída 2.0 binaural. Bypass se a rota não for fone ou se o Spatializer do sistema estiver ativo.
2	ParametricEqAudioProcessor (existente)	EQ do usuário + AutoEq do fone, que deve ter alvo de campo difuso/Harman compatível com a HRTF (ver 10.4).
3	Limitador true-peak (novo ou no EqDsp)	Lookahead de 1–2 ms, teto −1 dBTP. O espacial soma energia e pode passar de 0 dBFS.

10.2 Implementação: Kotlin ou C++?

O `EqDsp` atual em Kotlin funciona porque biquads são baratos. Para FFT por quadro recomendamos **C++ via JNI com NEON**: um único `process()` por buffer (não por amostra), dados em `FloatArray` diretos/ByteBuffer direto para evitar cópias. Biblioteca de FFT: PFFFT (BSD-like) ou KissFFT (BSD). Um protótipo em Kotlin puro é viável para validar a ideia (cerca de 38 MFLOP/s é aceitável no ART), mas o GC e as verificações de limite pesam em aparelhos simples.

10.3 Esqueleto do código

```
// Kotlin - camada Media3 (mesmo padrão do ParametricEqAudioProcessor)
@OptIn(UnstableApi::class)
class SpatialAudioProcessor(private val engine: SpatialEngine) : BaseAudioProcessor() {
  private var inCh = 2
  override fun onConfigure(f: AudioProcessor.AudioFormat): AudioProcessor.AudioFormat {
    if (f.encoding != C.ENCODING_PCM_FLOAT && f.encoding != C.ENCODING_PCM_16BIT)
      throw AudioProcessor.UnhandledAudioFormatException(f)
    if (!engine.enabled) return AudioProcessor.AudioFormat.NOT_SET   // = inativo/bypass
    inCh = f.channelCount
    engine.configure(f.sampleRate, inCh)          // aloca TUDO aqui, nunca no process
    return AudioProcessor.AudioFormat(f.sampleRate, 2, C.ENCODING_PCM_FLOAT)
  }
  override fun queueInput(input: ByteBuffer) {
    val frames = input.remaining() / (inCh * bytesPerSample())
    val out = replaceOutputBuffer(frames * 2 * 4)
    engine.process(input, out, frames)            // JNI: lê N canais, escreve 2 float
    input.position(input.limit()); out.flip()
  }
  override fun onFlush() = engine.reset()
}

// C++ - núcleo por quadro (hop = 512, janela = 1024, N = 2048)
void SpatialCore::processFrame() {
  window(inL, win, bufL, 1024); zeroPad(bufL, 2048);      // idem R
  fft.forward(bufL, XL); fft.forward(bufR, XR);
  for (int k = 0; k < 1025; ++k) {
    cf l = XL[k], r = XR[k];
    Pll[k] = a*Pll[k] + (1-a)*norm(l);
    Prr[k] = a*Prr[k] + (1-a)*norm(r);
    Plr[k] = a*Plr[k] + (1-a)*l*conj(r);
    float psi = abs(Plr[k]) / sqrtf(Pll[k]*Prr[k] + 1e-12f);
    float gl = sqrtf(Pll[k]), gr = sqrtf(Prr[k]);
    float th = atanf(TAN30 * (gl-gr) / (gl+gr+1e-12f));    // azimute original
    float D  = smoothErb(k, psi*psi);                       // fração direta
    Dir d    = rules.destination(k, th, D, transient);      // tabela 8.4
    d        = headTracker.rotate(d);                       // quaternion^-1
    cf src   = mid(l, r, gl, gr);
    const HrtfBin& h = hrtf.lookup(d, k);                   // grade pré-computada
    YL[k] = h.L * (sqrtf(D)*src) + ambL[k] * (sqrtf(1-D)*src);
    YR[k] = h.R * (sqrtf(D)*src) + ambR[k] * (sqrtf(1-D)*src);
    vis.accumulate(k, d, D, Pll[k]+Prr[k]);                 // para a UI
  }
  fft.inverse(YL, yl); fft.inverse(YR, yr);
  overlapAdd(yl, outL, 512); overlapAdd(yr, outR, 512);
}
```

10.4 Integração com o que o Tarkas já tem

- **AutoEq:** as curvas AutoEq levam o fone a um alvo (ex.: Harman). HRTFs medidas são normalizadas em campo livre ou difuso. Para não aplicar o "timbre da orelha" duas vezes, normalizar as HRTFs pelo **campo difuso** (média de potência em todas as direções = plana). Assim, um som frontal no centro soa quase igual ao estéreo original.
- **Spatializer do Android:** se `Spatializer.isEnabled()` e `isAvailable()` forem verdadeiros (Dolby/Samsung/Pixel do sistema), desligar o motor do Tarkas para não espacializar duas vezes, ou mostrar a escolha ao usuário. A AOSP diz que o Spatializer do sistema não processa estéreo, então para estéreo o motor do Tarkas é a única opção.
- **Detector de rota:** só ativar binaural em fone com fio, USB ou Bluetooth (`AudioDeviceInfo` TYPE_WIRED_HEADPHONES, TYPE_BLUETOOTH_A2DP, TYPE_USB_HEADSET, TYPE_BLE_HEADSET). Em alto-falante do celular, usar o modo "alargar" (seção 7.6).
- **GlobalEq / captura do sistema:** o mesmo núcleo C++ pode rodar no caminho de captura (AudioPlaybackCapture) usado pelo GlobalEqService, respeitando as limitações já documentadas (apps que bloqueiam captura).

- **Fontes web (YouTube Music / Spotify):** só se o áudio passar pelo player do Tarkas como PCM. Se tocar num WebView/SDK fechado, o motor não tem acesso ao sinal.

10.5 Visualizador "onde cada som está"

Como cada bin já tem (θ, φ, energia, D), a UI pode mostrar em tempo real uma esfera ou radar com pontos coloridos pela frequência (graves em vermelho, agudos em azul) e tamanho pela energia. Para não sobrecarregar a UI: agregar em ~40 bandas ERB, publicar 30 vezes por segundo por um buffer circular sem lock (o áudio escreve, a UI lê). Isso também serve de ferramenta de depuração do motor.

10.6 Parâmetros expostos ao usuário

Controle	Faixa	Efeito interno
Intensidade espacial	0–100%	Mistura entre estéreo original e renderização (dry/wet com compensação de nível).
Largura do palco	60°–220°	Fator w do azimute (seção 8.2).
Altura	0°–45°	e_{max} da heurística de elevação.
Sala	Estúdio / Sala / Arena	Reflexões e RT60 (equivalentes a Near/Mid/Far).
Âncora de voz	liga/desliga	Prende o centro 300 Hz–4 kHz à frente.
Perfil de HRTF	KU100, KEMAR, sujeitos SADIE	Troca a grade de HRTF (teste A/B para escolher a mais natural).
Head tracking	liga/desliga + recentrar	Usa TYPE_HEAD_TRACKER se disponível.

11. Loudness, testes e validação

11.1 Loudness

A Dolby recomenda para masters de música Atmos **−18 LKFS integrado** e **true-peak ≤ −1 dBTP** (medição ITU-R BS.1770). Plataformas de estéreo normalizam por volta de −14 LUFS. O motor deve **preservar a loudness**: medir em segundo plano (BS.1770 com filtro K, blocos de 400 ms) e aplicar uma correção lenta para que ligar ou desligar o espacial não pareça "mais alto = melhor", um viés clássico em testes A/B.

11.2 Testes objetivos (automatizáveis em JUnit, como o EqDspTest atual)

- **Reconstrução:** com o espacial em bypass interno (H = 1), saída = entrada atrasada em 1.024 amostras (erro < −100 dB).
- **Estimador de pan:** o teste da seção 8.5 como teste unitário: senos pan-eados em ±0,7, erro de pan < 0,02.
- **Coerência:** ruído descorrelacionado → ψ médio < 0,4; mono → ψ > 0,99.
- **Estabilidade:** silêncio, DC, ruído branco a 0 dBFS, impulsos, NaN injetado → saída finita e ≤ −1 dBTP.
- **Compatibilidade mono:** soma L+R da saída binaural sem cancelamentos > 6 dB em nenhuma banda ERB.
- **Desempenho:** microbenchmark do processFrame em aparelho de entrada (meta: < 0,5 ms por quadro de 10,7 ms).

11.3 Testes subjetivos

- **MUSHRA (ITU-R BS.1534):** comparar Tarkas Espacial, estéreo original, e uma referência (ex.: a mesma faixa em Atmos binaural da Apple Music) em escala 0–100.
- **Teste de localização:** sons de ruído rosa em 16 direções; o ouvinte aponta na tela. Mede erro angular e taxa de confusão frente/trás por perfil de HRTF.
- **Externalização:** escala de 0 (dentro da cabeça) a 3 (fora, longe) por preset de sala.
- **Fadiga:** sessões de 30 minutos; coloração e "ruído musical" do upmix costumam aparecer só em escuta longa.

12. Aspectos legais e de licenciamento

- **Marca:** "Dolby", "Dolby Atmos" e o símbolo duplo D são marcas registradas. O recurso do Tarkas deve ter nome próprio (ex.: "Tarkas Spatial" ou "Som 360 TARKAS") e não pode dizer que é Atmos, compatível com Atmos, nem usar o logo. Frases como "inspirado em" também são arriscadas em marketing. Recomenda-se não citar a marca.
- **Codecs:** decodificar E-AC-3, E-AC-3 JOC, TrueHD ou AC-4 exige licença. Usar o decodificador do próprio aparelho via MediaCodec é permitido (o fabricante já licenciou). Implementar decodificador próprio não é.
- **Patentes:** JOC, spatial coding e o panner alocêntrico têm patentes da Dolby. O motor descrito aqui usa técnicas de domínio público ou com literatura aberta há mais de 20 anos (VBAP de 1997, upmix por coerência de 2002–2004, HRTF, FDN). Mesmo assim, antes de lançamento comercial, recomenda-se uma busca de patentes por advogado, principalmente sobre upmix por frequência, que tem patentes de vários fabricantes.
- **Dados de HRTF:** cada base tem licença própria (tabela 7.2). Incluir créditos na tela de licenças do app, como já é feito para outros textos legais.
- **Alternativa aberta:** o IAMF (Alliance for Open Media) é um formato de áudio imersivo livre de royalties com decodificador aberto (libiamf) e extensão para Media3. Se o Tarkas um dia quiser tocar conteúdo imersivo "de verdade" com objetos, esse é o caminho sem licenciamento Dolby.

13. Plano de implementação em fases

Fase	Entregas	Critério de aceite
0 – Protótipo offline	Script Python/NumPy do algoritmo da seção 8 processando WAVs; escuta A/B; escolha da HRTF.	Pan estimado correto nos testes; ouvintes preferem o espacial em ≥ 60% dos trechos.
1 – Núcleo C++	STFT, estatísticas, máscaras ERB, HRTF por bin, OLA; testes de reconstrução e pan; JNI.	Testes da 11.2 passando; < 0,5 ms por quadro no aparelho de teste.
2 – Integração Media3	SpatialAudioProcessor na cadeia, detector de rota, bypass com Spatializer do sistema, limitador, compensação de loudness.	Troca de rota e de faixa sem cliques; falha do motor não derruba o player.
3 – Sala e UI	Presets de sala (reflexões + FDN), controles da 10.6, visualizador.	Externalização média ≥ 2 na escala 0–3.
4 – Multicanal e head tracking	Alto-falantes virtuais 5.1/7.1/7.1.4; TYPE_HEAD_TRACKER; recentrar.	Cena estável ao girar a cabeça; latência dentro do limite AOSP.
5 – Avançado (opcional)	Separação de fontes neural offline (ex.: modelos tipo Demucs/HTDemucs, licença MIT) ao carregar a faixa: voz, bateria, baixo e outros viram objetos reais; leitura de ADM/IAMF.	Melhoria significativa em MUSHRA sobre a Fase 3.

Sobre a separação neural: modelos como HTDemucs separam stems com qualidade muito superior ao upmix por coerência, mas custam ordens de grandeza mais (centenas de GFLOP por minuto de áudio). Em celular, isso só é viável como pré-processamento ao carregar a faixa (com cache), não em tempo real puro. O upmix espectral das seções 8.2–8.6 continua sendo o modo padrão, e a separação neural vira um modo "Premium" opcional.

14. Folha de fórmulas

Grandeza	Fórmula	Valor típico
ITD (Woodworth)	$(a/c)(\theta + \text{sen}\,\theta)$	máx. 0,656 ms = 31,5 amostras a 48 kHz
Limite de ITD de fase	$f \approx c / (2 \cdot \text{distância entre ouvidos})$	≈ 1,5 kHz
Inverso do quadrado	$\Delta L = -20 \cdot \log_{10}(r/r_0)$	−6 dB por dobro de distância
ERB	$24{,}7 \cdot (4{,}37 \cdot f/1000 + 1)$	133 Hz em 1 kHz; ~40 bandas no espectro
Pan seno/cosseno	$g_L = \cos((p+1)\pi/4)$, $g_R = \text{sen}((p+1)\pi/4)$	−3 dB no centro
Lei das tangentes	$\tan\theta / \tan\theta_0 = (g_L - g_R)/(g_L + g_R)$	$\theta_0 = 30°$
VBAP	$\mathbf{g} = \mathbf{p}^T\mathbf{L}^{-1}$, normalizado	9 multiplicações por atualização
Coerência	$\lvert\Phi_{LR}\rvert / \sqrt{(\Phi_{LL}\Phi_{RR})}$	1 = direto, ≈ 0 = ambiente
Pan estimado	$(4/\pi) \cdot \text{atan2}(\sqrt{\Phi_{RR}}, \sqrt{\Phi_{LL}}) - 1$	−1 … +1
Fração direta	$D = \max(\psi, \lambda)^\gamma$	$\gamma \approx 2$
Convolução linear na FFT	$L + M - 1 \le N$	1024 + 256 − 1 ≤ 2048
Resolução de bin	f_s / N	48000/2048 = 23,4 Hz
Taxa de quadros	f_s / H	93,75 quadros/s (10,7 ms)
Latência algorítmica	L / f_s	21,3 ms
Custo FFT real	$\approx 2{,}5 \cdot N \cdot \log_2 N$	≈ 56 kFLOP para N = 2048
Canais Ambisonics	$(N + 1)^2$	16 em 3ª ordem

15. Referências

Normas e documentos oficiais

- ETSI TS 102 366 – Digital Audio Compression (AC-3, Enhanced AC-3) Standard (inclui o container EMDF, Anexo H).
- ETSI TS 103 420 V1.2.1 (2018-10) – Backwards-compatible object audio carriage using Enhanced AC-3 (JOC, cláusula 6).
- ETSI TS 103 190-1 e -2 – Digital Audio Compression (AC-4) Standard.
- ITU-R BS.2051 – Advanced sound system for programme production (layouts de alto-falantes).
- ITU-R BS.2076 – Audio Definition Model (ADM). ITU-R BS.2088 – Long-form file format (BW64).
- ITU-R BS.2127 – Audio Definition Model renderer for advanced sound systems (implementação de referência EBU *libear*).
- ITU-R BS.1770 – Algoritmos de medição de loudness e true-peak. ITU-R BS.1534 – MUSHRA. ITU-R BS.1116 – pequenas degradações.
- SMPTE ST 2098-2 – Immersive Audio Bitstream (IAB).
- AES69-2015 – Spatial acoustic data file format (SOFA).
- Android Open Source Project – *Spatial audio and head tracking* e *Implementation of high-quality spatial audio and head tracking* (source.android.com/docs/core/audio/spatial).
- Alliance for Open Media – Immersive Audio Model and Formats (IAMF) v1.

Documentação de ferramentas (comportamento do Atmos)

- Steinberg – Cubase/Nuendo: *Object Audio*, *Exporting ADM Files*, *Authoring and Mixing for Dolby Atmos* (steinberg.help).
- Apple – Logic Pro User Guide: modos de renderização binaural Off/Near/Mid/Far no Dolby Atmos (support.apple.com).
- Avid – *Creating your Dolby Atmos mix with headphones* e *Advanced tips and tricks for mixing Dolby Atmos music* (avid.com).
- Dolby Hybrik – documentação de Dolby Atmos (docs.hybrik.com).

Artigos científicos e livros

- Rayleigh, Lord (1907). On our perception of sound direction. *Philosophical Magazine* 13. (Teoria Duplex.)
- Woodworth, R. S. (1938). *Experimental Psychology*. (Modelo de ITD.)
- Pratt, C. C. (1930). The spatial character of high and low tones. *J. Experimental Psychology*.
- Blauert, J. (1997). *Spatial Hearing: The Psychophysics of Human Sound Localization*. MIT Press.
- Glasberg, B. R.; Moore, B. C. J. (1990). Derivation of auditory filter shapes from notched-noise data. *Hearing Research* 47.
- Pulkki, V. (1997). Virtual sound source positioning using vector base amplitude panning. *JAES* 45(6).
- Brown, C. P.; Duda, R. O. (1998). A structural model for binaural sound synthesis. *IEEE Trans. Speech and Audio Processing* 6(5).
- Gardner, W. G.; Martin, K. D. (1995). HRTF measurements of a KEMAR. *JASA* 97(6).
- Algazi, V. R. et al. (2001). The CIPIC HRTF database. IEEE WASPAA.
- Avendano, C.; Jot, J.-M. (2002). Ambience extraction and synthesis from stereo signals for multi-channel audio up-mix. IEEE ICASSP.

- Avendano, C.; Jot, J.-M. (2004). A frequency-domain approach to multichannel upmix. *JAES* 52(7/8), pp. 740–749.
- Avendano, C. (2003). Frequency-domain source identification and manipulation in stereo mixes. IEEE WASPAA.
- Armstrong, C. et al. (2018). A perceptual evaluation of individual and non-individual HRTFs: a case study of the SADIE II database. *Applied Sciences* 8(11).
- Zotter, F.; Frank, M. (2019). *Ambisonics*. Springer (aberto). (Inclui MagLS e decodificação binaural.)
- Jot, J.-M.; Chaigne, A. (1991). Digital delay networks for designing artificial reverberators. AES Convention. (FDN.)
- Rouard, S.; Massa, F.; Défossez, A. (2023). Hybrid Transformers for Music Source Separation (HTDemucs). IEEE ICASSP.

Software aberto útil

- libmysofa (BSD) – leitura de SOFA. · EBU libear (Apache 2.0) – renderizador ADM BS.2127.
- Google Resonance Audio (Apache 2.0) – binaural Ambisonics. · Steam Audio (Apache 2.0) – HRTF, reflexões.
- PFFFT, KissFFT (BSD) – FFT com NEON. · libiamf (BSD-3) – decodificador IAMF.
- University of York – SADIE II Database (york.ac.uk/sadie-project).